

Practical 6

Sr. No	Practical Question	Type
1	Create a tuple named coordinates containing three numbers: 10.5, 20.0, and 15.2. Print the tuple, its length using len(), and its data type using type().	Easy
2	Given dimensions = (50, 100, 150, 200), use positive indexing to print the second element and negative indexing to print the last element.	Easy
3	Unpack the tuple point = (5, 12) into two variables x and y. Print both variables on separate lines.	Easy
4	Given ranks = (1, 2, 3, 2, 4, 2, 5), use the .count() method to find how many times 2 appears, and .index() to find the first position of 4.	Easy
5	Create a single-element tuple containing the string "Python". Ensure it is recognized as a tuple and not a string, then print its type.	Easy
6	Write a program attempting to change the first element of colors = ("red", "green", "blue") to "yellow". Use a try-except block to catch the TypeError and print "Tuples are immutable!".	Easy
7	Create a dictionary student = {"name": "Aarav", "age": 20, "course": "Computer Science"}. Print the value associated with the "course" key directly using square brackets [].	Easy

8	Given the dictionary <code>car = {"brand": "Toyota", "model": "Corolla"}</code> , add a new key-value pair <code>"year": 2022</code> . Print the updated dictionary.	Easy
9	Modify the value of <code>"price"</code> in <code>product = {"item": "Laptop", "price": 45000}</code> to 42000. Print the updated dictionary.	Easy
10	Given <code>profile = {"username": "coder123"}</code> , use an if-else statement with the <code>in</code> operator to check if <code>"email"</code> exists as a key. If present, print its value; otherwise, print <code>"Not Provided"</code> .	Easy
11	Given <code>inventory = {"apples": 10, "bananas": 5, "oranges": 8}</code> , iterate directly over the dictionary using a for loop to print each key along with its value using <code>inventory[key]</code> .	Moderate
12	Given a dictionary <code>user = {"name": "Siddharth", "role": "Admin"}</code> , check if the key <code>"email"</code> exists in the dictionary using the <code>in</code> operator and print <code>True</code> or <code>False</code> .	Moderate
13	Use the <code>.update()</code> method to merge <code>dict2 = {"c": 30, "d": 40}</code> into <code>dict1 = {"a": 10, "b": 20}</code> . Print the updated <code>dict1</code> .	Moderate
14	Given <code>person = {"name": "Riya", "age": 25, "city": "Mumbai"}</code> , use the <code>.pop()</code> method to remove <code>"city"</code> and print the popped value as well as the remaining dictionary.	Moderate
15	Given <code>session = {"user": "admin", "token": "xyz123", "status": "active"}</code> , use the <code>.popitem()</code> method to remove and return the last inserted key-value pair.	Moderate

16	Given settings = {"theme": "dark", "volume": 80}, use the del keyword to remove the "volume" key. Print the remaining dictionary.	Moderate
17	Given data = {"a": 1, "b": 2, "c": 3}, create a duplicate copy using .copy(). Clear all items from the original dictionary using .clear(). Verify that the copy remains intact.	Moderate
18	Write a program that converts a tuple num_tuple = (10, 20, 30) into a list, appends 40 to the list, converts it back into a tuple, and prints the updated tuple.	Moderate
19	Take a string from the user. Write a program that uses a loop and a dictionary to count the frequency of each character in the string. (e.g., "apple" → {'a': 1, 'p': 2, 'l': 1, 'e': 1}).	Moderate
20	Given a dictionary of student marks marks = {"Alice": 85, "Bob": 42, "Charlie": 78, "David": 35}, iterate through the student names and print whether each student "Passed" (mark >= 40) or "Failed".	Moderate

21. The Immutable GPS Log

A telemetry system records continuous location coordinates as tuples inside a list: route = [(18.52, 73.85), (19.07, 72.87), (12.97, 77.59)].

- Write a program that loops through route.
- Unpack each coordinate tuple into latitude and longitude.
- Print a formatted log string: "Checkpoint X -> Latitude: Y, Longitude: Z".
- Calculate the average latitude across all checkpoints using a loop.

22. The E-Commerce Shopping Cart Checkout

An online store tracks items selected by a user in a dictionary where the key is the product name and the value is its price: `cart = {"Shirt": 800, "Jeans": 1500, "Shoes": 2200, "Hat": 300}`.

- Ask the user if they have a promo code.
- If the promo code is "DISCOUNT10", apply a 10% discount to all items by updating the values in the dictionary.
- Calculate the final total bill by looping through the keys of `cart` and adding up `cart[key]`.
- If the total bill exceeds 3000, print "Eligible for Free Shipping!", otherwise print "Standard Shipping Applied".

23. The Bank Account Manager (Dictionary + Nested Keys)

A bank branch stores customer data using account numbers as keys: `accounts = {1001: {"name": "Priya", "balance": 5000}, 1002: {"name": "Rahul", "balance": 12000}}`.

- Ask the user to enter their account number.
- Use the `in` operator to check if the account exists.
- If it exists, ask whether they want to "deposit" or "withdraw", along with the amount.
- Update the "balance" key inside the nested dictionary directly using `accounts[acc_num]["balance"]`.
- If a withdrawal exceeds the available balance, cancel the transaction and print "Insufficient Funds".

24. The Hotel Room Booking Reset

A boutique hotel manages room statuses using a dictionary: `rooms = {101: "Occupied", 102: "Vacant", 103: "Occupied", 104: "Under Maintenance", 105: "Occupied"}`.

- Write a program that loops through the room numbers (keys in `rooms`).
- For every room where `rooms[room_number] == "Occupied"`, update its status to "Needs Cleaning".
- Maintain a counter variable to count how many total rooms were changed to "Needs Cleaning".
- Print the updated `rooms` dictionary and the total count.

25. The Contact Book Search & Purge

You are designing the backend for a mobile phonebook app: `contacts = {"Amit": "9876543210", "Neha": "9123456789", "Suresh": "9988776655"}`.

- Ask the user for a contact name to search.
- If the name exists in `contacts` (using `in`), ask if they want to update the phone number or delete the contact.
- If they choose update, take a new number input and update the entry using `.update()`.
- If they choose delete, use `.pop()` to remove the key and print "Contact deleted successfully."
- If the name does not exist, ask if they want to add it as a new contact.

26. The Library Inventory & Return System

A school library tracks available copies of books: `library = {"Python Basics": 4, "Data Structures": 2, "Web Dev": 0}`.

- A student wants to borrow "Web Dev". Check if the book exists in `library` using `in` and if its count `library["Web Dev"]` is greater than 0.
- If the count is 0, print "Book currently out of stock."
- If a student returns 2 copies of "Python Basics", update the inventory count by adding 2 to `library["Python Basics"]`.
- Automatically delete any book entry from `library` using `del` if its inventory count ever drops to 0.

27. The Voting System Duplicate Filter (Tuple Keys)

An election poll uses voter records where keys are tuples containing (`voter_id`, `region_code`) to prevent duplicate voting: `voter_registry = {(101, "WEST"): "Voted", (102, "EAST"): "Voted"}`.

- A new voter arrives with `voter_id = 101` and `region_code = "WEST"`.
- Create a tuple `current_voter = (101, "WEST")`.
- Check if `current_voter` exists in `voter_registry` using the `in` operator.
- If it exists, print "Error: Voter has already cast a vote!"
- If it does not exist, add `current_voter` to `voter_registry` with value "Voted" and print "Vote recorded successfully!"

28. The Employee Shift Expirations (`popitem()` simulation)

A warehouse assigns task queues to workers using a dictionary ordered by shift priority: `task_queue = {"Task1": "Unload Truck", "Task2": "Inspect Boxes", "Task3": "Label Packages"}`.

- Write a while loop that processes tasks one by one from the end of the queue using `.popitem()`.
- In each iteration, unpack the popped tuple into `task_id` and `task_desc`, then print "Processing and removing: [task_id] -> [task_desc]".
- Stop the loop when `len(task_queue) == 0` and print "All shift tasks completed!"

29. The Gradebook Merging & Cleaning Strategy

Two academic branches merge their mid-term score sheets: `branch1 = {"StudentA": 78, "StudentB": 85, "StudentC": 90}` `branch2 = {"StudentB": 88, "StudentD": 92}` (Note: *StudentB* took tests in both branches).

- Create a copy of `branch1` named `final_gradebook`.
- Loop through the keys of `branch2`. If a student already exists in `final_gradebook`, update their score only if the new score in `branch2` is higher.
- If the student does not exist in `final_gradebook`, add them.
- Print the merged `final_gradebook`.

30. The Smart Pantry Restock Alert

A smart refrigerator tracks grocery levels in percentages: `pantry = {"Milk": 15, "Eggs": 80, "Butter": 5, "Cheese": 50, "Juice": 10}`.

- Create two empty lists: `critical_restock` and `sufficient_stock`.
- Iterate through the items in `pantry` using a for loop over its keys.
- If `pantry[item]` is **below 20%**, add a tuple (`item`, `pantry[item]`) to `critical_restock`.
- If `pantry[item]` is **20% or higher**, append the item name to `sufficient_stock`.
- Print both lists clearly formatted for the user's shopping display.